Tema 6.1: Introducción a las Hebras en Java

Pablo Gordillo (pabgordi@ucm.es)

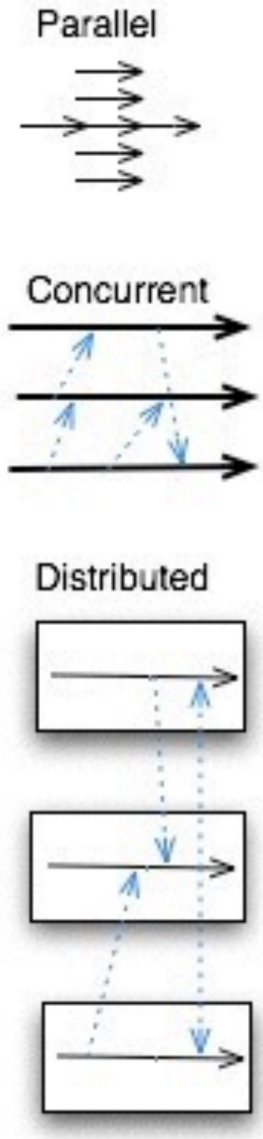
Departamento de Sistemas Informáticos y Computación

Facultad de Informática



(Basado en el material creado por Puri Arenas, Jesús Correas y Samir Genaim)

Paralelo, Concurrente, Distribuido...



- **Paralelismo.** Se trata de coger un programa y hacer que se ejecute más rápido, dividiendo el código en partes que se pueden ejecutar simultáneamente.
- **Concurrencia.** Se trata de sistemas que tienen varias partes que están diseñadas para ejecutar simultáneamente, no porque lo hace más rápido, sino porque es una parte necesaria de su funcionalidad.
- **Distribución.** Se trata de sistemas compuestos de varias partes físicas conectadas por una red de comunicación, cada una ejecutando posiblemente diferentes programas, etc.

Hebras y Procesos

- Manera de implementar concurrencia/paralelismo
- Un programa puede tener varios procesos que se ejecutan "simultáneamente", cada uno tiene su propia memoria, archivos, etc. Cada proceso es como si fuera un programa separado.
- Cada proceso puede tener varias hebras que comparten sus recursos (memoria, etc.)
- Norlamente, un programa tiene como mínimo un proceso y cada proceso tiene como mínimo una hebra.
- En Java, cuando se ejecuta un programa, se crea un proceso con una hebra que ejecuta el método **main**.
- Nos centraremos en los hilos (threads) de Java.

Qué significa "Ejecutando Simultáneamente (en Paralelo)"?

- Esto depende de muchos factores. Por ejemplo, si tenemos n hebras ejecutando en paralelo en una plataforma con m >= n CPUs/Cores, podemos pensar que realmente están ejecutando en paralelo, aunque no tiene por que...
- Sin embargo, normalmente tenemos hebras (en todos los programas) más que CPUs/Cores.
- Un **gestor de hebras** (a nivel del sistema operativo, la JVM, hardware, etc.) alterna la ejecución de las hebras, es decir, se ejecuta un poco de cada hebra para que parezca que están ejecutandose en paralelo.
- No existe ningún orden de ejecución de los procesos, sino que se alternarán dependiendo de las capacidades del procesador y del planificador.

Ejemplo 1

- Ya hemos utilizado hebras en Swing, ya que la interfaz gráfica se ha ejecutado en una hebra distinta. En este ejemplo tenemos **2 hebras**

```
public class Hebras extends JFrame {
  ...
  public void initGUI() {
    ...
    JButton boton1 = new JButton("Message");
    boton1.addActionListener(new ActionListener() {
      @Override
      public void actionPerformed(ActionEvent arg0) {
        JOptionPane.showMessageDialog(null,"INFORMACION");
      }
    });
    ...
  }
  public static void main(String[] args) throws InterruptedException {
    SwingUtilities.invokeLater(new Runnable() {
      public void run() { new Hebras(); }
    });
    while (true) {
      cont++;
      Thread.sleep(5000);
      System.out.println(cont);
    }
  }
}
```

Crear y Empezar una Hebra

- Para crear una hebra podemos implementar una clase que extienda a **java.lang.Thread** y:
  - Sobreescribir el método **run()** (que es como el main() de la hebra).
  - Lanzar la hebra utilizando el método **start()**.

```
public class Hebra1 extends Thread {
  private String id;
  public Hebra1(String id) {
    this.id = id;
  }
  public void run() {
    int i=0;
    while (i < 100) {
      System.out.println (this.id + ": " + i);
      i++;
    }
  }
}
```

Crear y Empezar una Hebra

- Para crear una hebra podemos implementar una clase que extienda a **java.lang.Thread** y:
  - Sobreescribir el método **run()** (que es como el main() de la hebra).
  - Lanzar la hebra utilizando el método **start()**.

```
public class Hebra1 extends Thread {
  private String id;
  public Hebra1(String id) {
    this.id = id;
  }
  public void run() {
    int i=0;
    while (i < 100) {
      System.out.println (this.id + ": " + i);
      i++;
    }
  }
}
```

Crear y Empezar una Hebra

- Para crear una hebra podemos implementar una clase que extienda a **java.lang.Thread** y:
  - Sobreescribir el método **run()** (que es como el main() de la hebra).
  - Lanzar la hebra utilizando el método **start()**.

```
public class Hebra1 extends Thread {
  private String id;
  public Hebra1(String id) {
    this.id = id;
  }
  public void run() {
    int i=0;
    while (i < 100) {
      System.out.println (this.id + ": " + i);
      i++;
    }
  }
}
```

Todas las instrucciones de run se ejecutan en una misma hebra

Crear y Empezar una Hebra

- Para crear una hebra podemos implementar una clase que extienda a **java.lang.Thread** y:
  - Sobreescribir el método **run()** (que es como el main() de la hebra).
  - Lanzar la hebra utilizando el método **start()**.

```
public class Hebra1 extends Thread {
  private String id;
  public Hebra1(String id) {
    this.id = id;
  }
  public void run() {
    int i=0;
    while (i < 100) {
      System.out.println (this.id + ": " + i);
      i++;
    }
  }
}
...
public static void main(String[] args) {
  Thread h1 = new Hebra1("Hebra 1");
  Thread h2 = new Hebra1("Hebra 2");
  h1.start ();
  h2.start ();
}
```

start inicia la hebra y comienza la ejecución de run de esa hebra

Crear y Empezar una Hebra

- Para crear una hebra podemos implementar una clase que extienda a **java.lang.Thread** y:
  - Sobreescribir el método **run()** (que es como el main() de la hebra).
  - Lanzar la hebra utilizando el método **start()**.

```
public class Hebra1 extends Thread {
  private String id;
  public Hebra1(String id) {
    this.id = id;
  }
  public void run() {
    int i=0;
    while (i < 100) {
      System.out.println (this.id + ": " + i);
      i++;
    }
  }
}
...
public static void main(String[] args) {
  Thread h1 = new Hebra1("Hebra 1");
  Thread h2 = new Hebra1("Hebra 2");
  h1.start();
  h2.start();
}
```

start inicia la hebra y comienza la ejecución de run de esa hebra

Quien llama a start sigue su ejecución en paralelo

Crear y Empezar una Hebra

- Para crear una hebra podemos implementar una clase que extienda a **java.lang.Thread** y:
  - Sobreescribir el método **run()** (que es como el main() de la hebra).
  - Lanzar la hebra utilizando el método **start()**.

```
public class Hebra1 extends Thread {
  private String id;
  public Hebra1(String id) {
    this.id = id;
  }
  public void run() {
    int i=0;
    while (i < 100) {
      System.out.println (this.id + ": " + i);
      i++;
    }
  }
}
...
public static void main(String[] args) {
  Thread h1 = new Hebra1("Hebra 1");
  Thread h2 = new Hebra1("Hebra 2");
  h1.start ();
  h2.start ();
}
```

IMPORTANTE: El objeto es solo para controlar la hebra, pero no es la hebra.

Crear y Empezar una Hebra

- Para crear una hebra podemos implementar una clase que extienda a **java.lang.Thread** y:
  - Sobreescribir el método **run()** (que es como el main() de la hebra).
  - Lanzar la hebra utilizando el método **start()**.

```
public class Hebra1 extends Thread {
  private String id;
  public Hebra1(String id) {
    this.id = id;
  }
  public void run() {
    int i=0;
    while (i < 100) {
      System.out.println (this.id + ": " + i);
      i++;
    }
  }
}
...
public static void main(String[] args) {
  Thread h1 = new Hebra1("Hebra 1");
  Thread h2 = new Hebra1("Hebra 2");
  h1.start();
  h2.start();
}
```

Thread: #0
Desc: main está ejecutandose en esta hebra
State: RUNNABLE

Crear y Empezar una Hebra

- Para crear una hebra podemos implementar una clase que extienda a **java.lang.Thread** y:
  - Sobreescribir el método **run()** (que es como el main() de la hebra).
  - Lanzar la hebra utilizando el método **start()**.

```
public class Hebra1 extends Thread {
  private String id;
  public Hebra1(String id) {
    this.id = id;
  }
  public void run() {
    int i=0;
    while (i < 100) {
      System.out.println (this.id + ": " + i);
      i++;
    }
  }
}
...
public static void main(String[] args) {
  Thread h1 = new Hebra1("Hebra 1");
  Thread h2 = new Hebra1("Hebra 2");
  h1.start();
  h2.start();
}
```

Thread: #0
Desc: main está ejecutandose en esta hebra
State: RUNNABLE

Thread: #1
Desc: hebra correspiente a h1.
State: NEW

Crear y Empezar una Hebra

- Para crear una hebra podemos implementar una clase que extienda a **java.lang.Thread** y:
  - Sobreescribir el método **run()** (que es como el main() de la hebra).
  - Lanzar la hebra utilizando el método **start()**.

```
public class Hebra1 extends Thread {
  private String id;
  public Hebra1(String id) {
    this.id = id;
  }
  public void run() {
    int i=0;
    while (i < 100) {
      System.out.println (this.id + ": " + i);
      i++;
    }
  }
}
...
public static void main(String[] args) {
  Thread h1 = new Hebra1("Hebra 1");
  Thread h2 = new Hebra1("Hebra 2");
  h1.start();
  h2.start();
}
```

Thread: #0
Desc: main está ejecutandose en esta hebra
State: RUNNABLE

Thread: #1
Desc: hebra correspiente a h1.
State: NEW

Thread: #2
Desc: hebra correspiente a h2.
State: NEW

Crear y Empezar una Hebra

- Para crear una hebra podemos implementar una clase que extienda a **java.lang.Thread** y:
  - Sobreescribir el método **run()** (que es como el main() de la hebra).
  - Lanzar la hebra utilizando el método **start()**.

```
public class Hebra1 extends Thread {
  private String id;
  public Hebra1(String id) {
    this.id = id;
  }
  public void run() {
    int i=0;
    while (i < 100) {
      System.out.println (this.id + ": " + i);
      i++;
    }
  }
}
...
public static void main(String[] args) {
  Thread h1 = new Hebra1("Hebra 1");
  Thread h2 = new Hebra1("Hebra 2");
  h1.start();
  h2.start ();
}
```

Thread: #0
Desc: main está ejecutandose en esta hebra
State: RUNNABLE

Thread: #1
Desc: hebra correspiente a h1.
State: RUNNABLE

Thread: #2
Desc: hebra correspiente a h2.
State: NEW

Crear y Empezar una Hebra

- Para crear una hebra podemos implementar una clase que extienda a **java.lang.Thread** y:
  - Sobreescribir el método **run()** (que es como el main() de la hebra).
  - Lanzar la hebra utilizando el método **start()**.

```
public class Hebra1 extends Thread {
  private String id;
  public Hebra1(String id) {
    this.id = id;
  }
  public void run() {
    int i=0;
    while (i < 100) {
      System.out.println (this.id + ": " + i);
      i++;
    }
  }
}
...
public static void main(String[] args) {
  Thread h1 = new Hebra1("Hebra 1");
  Thread h2 = new Hebra1("Hebra 2");
  h1.start ();
  h2.start();
}
```

Thread: #0
Desc: main está ejecutandose en esta hebra
State: RUNNABLE

Thread: #1
Desc: hebra correspiente a h1.
State: RUNNABLE

Thread: #2
Desc: hebra correspiente a h2.
State: RUNNABLE

Crear y Empezar una Hebra

- Para crear una hebra podemos implementar una clase que extienda a **java.lang.Thread** y:
  - Sobreescribir el método **run()** (que es como el main() de la hebra).
  - Lanzar la hebra utilizando el método **start()**.

```
public class Hebra1 extends Thread {
  private String id;
  public Hebra1(String id) {
    this.id = id;
  }
  public void run() {
    int i=0;
    while (i < 100) {
      System.out.println (this.id + ": " + i);
      i++;
    }
  }
}
...
public static void main(String[] args) {
  Thread h1 = new Hebra1("Hebra 1");
  Thread h2 = new Hebra1("Hebra 2");
  h1.start ();
  h2.start ();
}
```

Thread: #0
Desc: main está ejecutandose en esta hebra
State: TERMINATED

Thread: #1
Desc: hebra correspiente a h1.
State: RUNNABLE

Thread: #2
Desc: hebra correspiente a h2.
State: RUNNABLE

Crear y Empezar una Hebra

- Para crear una hebra podemos implementar una clase que extienda a **java.lang.Thread** y:
  - Sobreescribir el método **run()** (que es como el main() de la hebra).
  - Lanzar la hebra utilizando el método **start()**.

```
public class Hebra1 extends Thread {
  private String id;
  public Hebra1(String id) {
    this.id = id;
  }
  public void run() {
    int i=0;
    while (i < 100) {
      System.out.println (this.id + ": " + i);
      i++;
    }
  }
}
...
public static void main(String[] args) {
  Thread h1 = new Hebra1("Hebra 1");
  Thread h2 = new Hebra1("Hebra 2");
  h1.start ();
  h2.start ();
}
```

Thread: #0
Desc: main está ejecutandose en esta hebra
State: TERMINATED

Thread: #1
Desc: hebra correspiente a h1.
State: TERMINATED

Thread: #2
Desc: hebra correspiente a h2.
State: RUNNABLE

Crear y Empezar una Hebra

- Para crear una hebra podemos implementar una clase que extienda a **java.lang.Thread** y:
  - Sobreescribir el método **run()** (que es como el main() de la hebra).
  - Lanzar la hebra utilizando el método **start()**.

```
public class Hebra1 extends Thread {
  private String id;
  public Hebra1(String id) {
    this.id = id;
  }
  public void run() {
    int i=0;
    while (i < 100) {
      System.out.println (this.id + ": " + i);
      i++;
    }
  }
}
...
public static void main(String[] args) {
  Thread h1 = new Hebra1("Hebra 1");
  Thread h2 = new Hebra1("Hebra 2");
  h1.start ();
  h2.start ();
}
```

Thread: #0
Desc: main está ejecutandose en esta hebra
State: TERMINATED

Thread: #1
Desc: hebra correspiente a h1.
State: TERMINATED

Thread: #2
Desc: hebra correspiente a h2.
State: TERMINATED

Creación de una hebra en Java

- Si queremos utilizar hebras en una clase que extiende a otra necesitamos utilizar la interfaz **Runnable**, ya que en Java no hay herencia múltiple

```
public class A extends B implements Runnable {
  private String id;
  public A(String id) {
    this.id = id;
  }
  @Override
  public void run() {
    int i=0;
    while (i<100) {
      System.out.println (id + ": " + i);
      i++;
    }
  }
  public static void main(String args[]) {
    Thread t1 = new Thread(new A("hebra1"));
    Thread t2 = new Thread(new A("hebra2"));
    t1.start();
    t2.start();
```

```
public interface Runnable {
  public abstract void run();
}
```

La clase Thread implementa a la Interfaz Runnable

Hebras usando Clases Anónimas

```
public class ThreadEx3 {
  public static void main(String args[]) {
    new Thread() {
      public void run() {
        int i = 0;
        while (i < 100) {
          System.out.println ("T1: " + i);
          i++;
        }
      }
    }.start ();
    new Thread(new Runnable() {
      public void run() {
        int i = 0;
        while (i < 100) {
          System.out.println ("T2: " + i);
          i++;
        }
      }
    }).start ();
    System.out.println ("Main is done!");
  }
}
```

1. Creamos una instancia de una clase anónima que extiende de **Thread**
2. Llamamos a **start** de la clase anónima

Hebras usando Clases Anónimas

```
public class ThreadEx3 {
  public static void main(String args[]) {
    new Thread() {
      public void run() {
        int i = 0;
        while (i < 100) {
          System.out.println ("T1: " + i);
          i++;
        }
      }
    }.start();
    new Thread(new Runnable() {
      public void run() {
        int i = 0;
        while (i < 100) {
          System.out.println ("T2: " + i);
          i++;
        }
      }
    }).start();
    System.out.println ("Main is done!");
  }
}
```

1. Creamos una instancia de una clase anónima que implementa **Runnable** y se pasa como parámetro a la constructora de Thread
2. Llamamos a **start** de la clase anónima

Terminación de un programa con hebras

- Un programa con hebras termina cuando todas sus hebras terminan.
- Una hebra termina cuando termina la ejecución de su método **run**.

```
public class Ex1 implements Runnable {
  private String id;
  public Ex1(String id) {this.id = id;}
  @Override
  public void run() {
    int i=0;
    while (true) {
      System.out.println (id + ": " + i);
      i++;
    }
  }
}
```

```
public class Ex2 implements Runnable {
  private String id;
  public Ex2(String id) {this.id = id;}
  @Override
  public void run() {
    int i=0;
    while (i < 100) {
      System.out.println (id + ": " + i);
      i++;
    }
  }
}
```

```
public static void main(String[] args) {
  Thread t1 = new Thread(new Ex1("A"));
  Thread t2 = new Thread(new Ex2("B"));
  t1.start();
  t2.start();
}
```

Terminación de un programa con hebras

- Un programa con hebras termina cuando todas sus hebras terminan.
- Una hebra termina cuando termina la ejecución de su método **run**.

```
public class Ex1 implements Runnable {
  private String id;
  public Ex1(String id) {this.id = id;}
  @Override
  public void run() {
    int i=0;
    while (true) {
      System.out.println (id + ": " + i);
      i++;
    }
  }
}
```

```
public class Ex2 implements Runnable {
  private String id;
  public Ex2(String id) {this.id = id;}
  @Override
  public void run() {
    int i=0;
    while (i < 100) {
      System.out.println (id + ": " + i);
      i++;
    }
  }
}
```

```
public static void main(String[] args) {
  Thread t1 = new Thread(new Ex1("A"));
  Thread t2 = new Thread(new Ex2("B"));
  t1.start();
  t2.start();
}
```

- La hebra main **termina**
- La hebra 2 **termina**
- **El programa no termina**

El estado de una hebra

Se puede consultar el estado de una hebra **t** usando **t.getState()**. El estado de una hebra puede ser:

- **NEW:** La hebra se ha creado pero no ha empezado todavía su ejecución.
- **RUNNABLE:** La hebra está ejecutándose o esperando a que el gestor de hebras la deje empezar la ejecución.
- **BLOCKED:** Bloqueada, esperando para obtener un **monitor lock**.
- **WAITING:** Esperando un **tiempo indefinido** hasta que "ocurra algo" que la despierte.
- **TIMED_WAITING:** Esperando un **tiempo fijo**, o hasta que "ocurra algo" que la despierte.
- **TERMINATED:** Hebra terminada.

El estado de una hebra

```
public class Ex4 extends Thread {
  private String id;

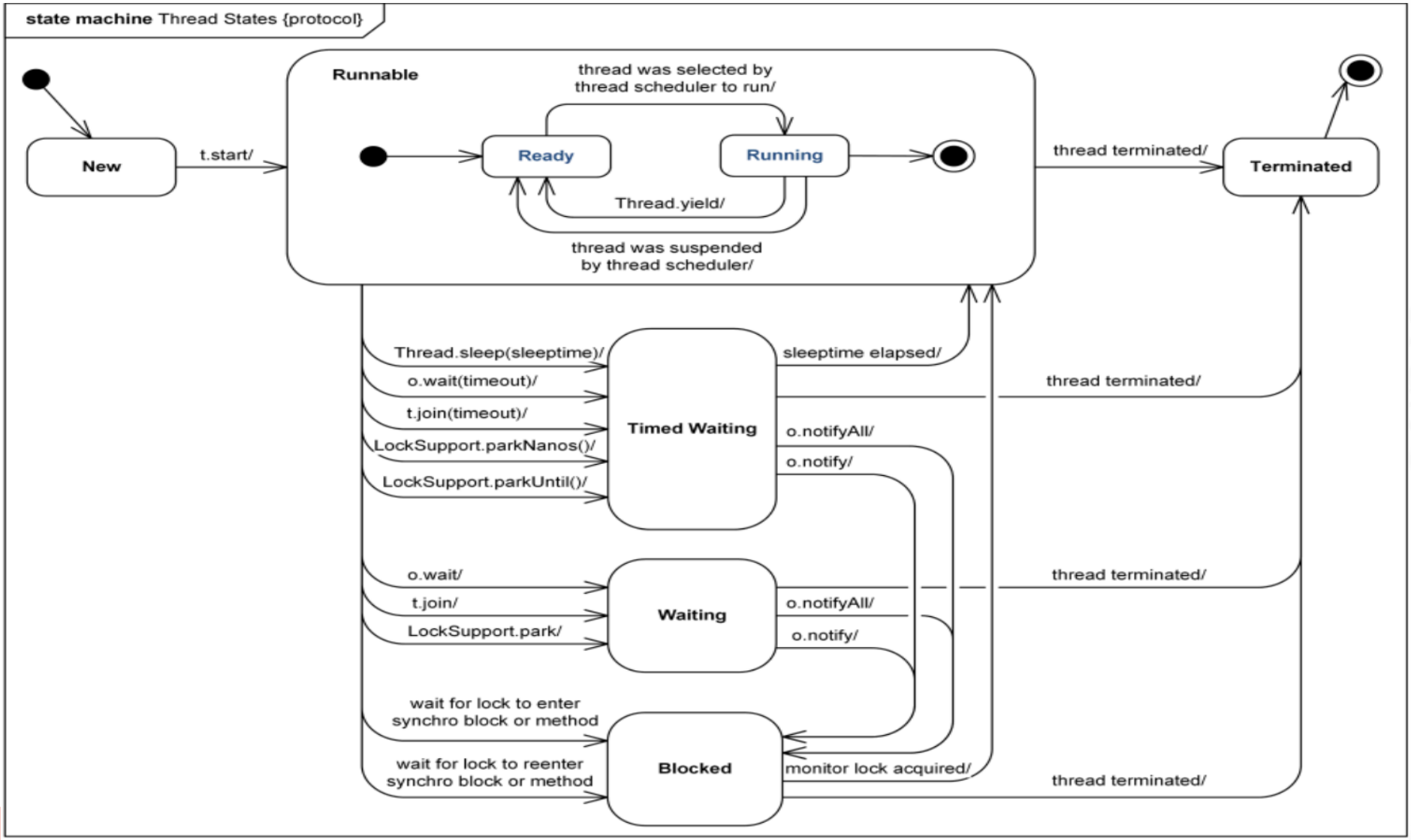
  Ex4(String id){
    this.id = id;
  }

  public void run(){
    int i = 0;
    while(i<100) {
      System.out.println (id+":"+i);
    }
  }
}

public static void main(String[] args)
                throws InterruptedException {
  Thread t1 = new Ex4("A");
  Thread t2 = new Ex4("B");
  t1.start();
  t2.start();
  t1.join();
  t2.join();
  System.out.println ("Main is done!");
}
```

1. La hebra que ejecuta **t.join()** se bloquea hasta que la hebra **t** termine.
2. Mientras está esperando su estado es **WAITING**.
3. Se puede interrumpir la hebra bloqueada, en ese caso se lanza una excepción.
4. **t.join(n)** espera **n** milisegundos y **t.join(n,m)** espera **n** milisegundos y **m** nanosegundos. Mientas está esperando su estado sería **TIMED_WAITING**

El estado de una hebra



El estado de una hebra

```
public class Ejemplo1 implements Runnable {
  private String id;
  public Ejemplo1(String id) {
    this.id = id;
  }
  @Override
  public void run() {
    int i=0;
    while (i<5) {
      System.out.println (id+": "+i);
      i++;
    }
  }
  public static void main(String args[]) throws InterruptedException {
    Thread t1 = new Thread( new Ejemplo1("Hebra 1") );
    Thread t2 = new Thread( new Ejemplo1("Hebra 2") );
    t1.start();
    System.out.println ("Estado Hebra 1: " + t1.getState());
    System.out.println ("Estado Hebra 2: " + t2.getState());
    t1.join();
    System.out.println ("Estado Hebra 1: " + t1.getState());
    t2.start();
    System.out.println ("Estado Hebra 2: " + t2.getState());
    t2.join();
    System.out.println ("Estado Hebra 2: " + t2.getState());
    System.out.println ("Main is done!")
  }
}
```

El estado de una hebra

```
public class Ejemplo1 implements Runnable {
  private String id;
  public Ejemplo1(String id) {
    this.id = id;
  }
  @Override
  public void run() {
    int i=0;
    while (i<5) {
      System.out.println (id+": "+i);
      i++;
    }
  }
  public static void main(String args[]) throws InterruptedException {
    Thread t1 = new Thread( new Ejemplo1("Hebra 1") );
    Thread t2 = new Thread( new Ejemplo1("Hebra 2") );
    t1.start();
    System.out.println ("Estado Hebra 1: " + t1.getState()); t1 RUNNABLE
    System.out.println ("Estado Hebra 2: " + t2.getState()); t2 NEW
    t1.join(); main WAITING
    System.out.println ("Estado Hebra 1: " + t1.getState()); t1 TERMINATED
    t2.start();
    System.out.println ("Estado Hebra 2: " + t2.getState()); t2 RUNNABLE
    t2.join(); main WAITING
    System.out.println ("Estado Hebra 2: " + t2.getState()); t2 TERMINATED
    System.out.println ("Main is done!") main TERMINATED
  }
}
```

Calcular una Función en Paralelo

```
public abstract class Function extends Thread {

  protected long result ;

  protected abstract void apply() throws InterruptedException ;

  public long getRes() {
    return result ;
  }

  public void run() {
    try {
      apply();
    } catch (InterruptedException e) {
      System.err.println ("Something went wrong");
    }
  }
}
```

Fact(n), Fib(n), F(n) = Fib(n)+Fact(n)

```
public class Fact extends Function {
  private long n;

  Fact(long n) { this.n = n; }

  protected void apply(){
    long r = 1;
    for (long i = 1; i<=n; i++) r*=i;
    this.result = r;
  }
}
```

```
public class Fib extends Function {
  private long n;

  Fib(long n) {this.n = n; }

  protected void apply() {
    long a = 0, b = 1, i = 0, c;
    for (int i = 1; i < n; i++){
      c = a+b; a = b; b = c;
    }
    this.result = b;
  }
}
```

```
public class F extends Function {
  private long n;

  F(long n) { this.n = n; }

  protected void apply() throws{
    Function f = new Fact(n);
    Function g = new Fib(n);
    f.start();
    g.start();
    f.join();
    g.join();
    this.result = f.getRes()+g.getRes();
  }
}
```

Calcular una Función en Paralelo

- Podemos medir el tiempo de la ejecución

```
public class Main {

  public static void main(String[] args) throws InterruptedException {

    Scanner sc = new Scanner(System.in);
    System.out.print ("Introduce an integer : ");
    long n = sc.nextLong();

    long startTime = System.currentTimeMillis( );

    Function t = new F(n);
    t.start ();
    t.join ();

    long taskTimeMs = System.currentTimeMillis( ) - startTime;

    System.out.println ("Res: " + t.getRes());
    System.out.println ("Time: " + taskTimeMs +"ms");
    sc.close ();
  }
}
```

Gestión de hebras

- **sleep(int m), sleep(int m, int n)**: La hebra duerme **m** milisegundos (y **n** nanosegundos) – el gestor de las hebras no la programa para ejecutar durante ese tiempo. Si estado sería **TIMED_WAITING**.
- Cada hebra tiene prioridad asignada. El gestor de hebra usa esas prioridades para elegir la siguiente hebra a ejecutar.
  1. **t.getPriority()** devuelve la prioridad de t
  2. **t.setPriority(int p)** cambia la prioridad de **t** a **p**. Para fijar la prioridad se pueden usar las constantes:
     MIN_PRIORITY(=1), MAX_PRIORITY(=10), NORM_PRIORITY(=5)
- **Thread.yield()** avisa al gestor de hebras que la hebra **actual** quiere ceder la CPU (es decir, dejar a otras hebras ejecutarse). **El gestor de hebras puede ignorar la petición.**
- **La hebra de gestión de eventos de Swing se ejecuta con una prioridad 6.**

Interrumpir a una Hebra

- Es un mecanismo de comunicación entre hebras.
- Una hebra puede interrumpir a otra, se puede ver como una notificación.
- **t.interrupt()**: interrumple la hebra **t** – el efecto depende del estado de **t** (lo veremos con ejemplos). Si el estado de **t** es RUNNABLE, el estado de interrupción de **t** cambia a **true**.
- **Thread.interrupted()**: devuelve **true** si **la hebra que invoca a Thread.interrupted()** ha sido interrumpida y cambia el estado de interrupción a **false**.
- **t.isInterrupted()**: devuelve **true** si la hebra **t** ha sido interrumpida. En este caso no cambia el estado de interrupción de **t**.

El efecto de t.interrupt()

- **NEW**: no pasa nada.
- **RUNNABLE**: cambia el estado de interrupción a true.
- **BLOCKED**: cambia el estado de interrupción a true.
- **WAITING**: lanza una excepción.
- **TIMED_WAITING**: lanza una excepción.
- **TERMINATED**: no pasa nada.

Interrumpir a una Hebra

```
public class IntEx1 implements Runnable {

  private String id;
  public IntEx1(String id) { this.id = id; }

  @Override
  public void run() {
    int i=0;
    while ( !Thread.interrupted() ) {
      System.out.println (id + ": " + i);
      i++;
    }
  }

  public static void main(String args[]) throws InterruptedException {
    Thread t1 = new Thread( new IntEx1("TEx1") );
    t1.start();
    Thread.sleep(5000);
    t1.interrupt();
  }
}
```

El bucle termina cuando la ejecución de la hebra se interrumpa

Interrumpir a una Hebra

```
public class IntEx1 implements Runnable {

  private String id;
  public IntEx1(String id) { this.id = id; }

  @Override
  public void run() {
    int i=0;
    while ( !Thread.interrupted() ) {
      System.out.println (id + ": " + i);
      i++;
    }
  }

  public static void main(String args[]) throws InterruptedException {
    Thread t1 = new Thread( new IntEx1("TEx1") );
    t1.start();
    Thread.sleep(5000);
    t1.interrupt();
  }
}
```

El bucle termina cuando la ejecución de la hebra se interrumpa

Interrumpir a una Hebra

```
public class IntEx2 implements Runnable {
  private String id;

  public IntEx2(String id) { this.id = id; }

  @Override
  public void run() {
    int i=0;
    while ( !Thread.interrupted() ) {
      System.out.println (id+": "+i);
      try {
        Thread.sleep(3000);
      }
      catch (InterruptedException e) {
        System.out.println ("Int while sleeping");

      }
      i++;
    }
  }

  public static void main(String args[]) throws InterruptedException {
    Thread t1 = new Thread( new IntEx2("TEx1") );
    t1.start();

    t1.interrupt();
  }
}
```

No se muestra nada. La hebra se interrumpe antes de la condición del bucle no entrando en el while

Interrumpir a una Hebra

```
public class IntEx2 implements Runnable {
  private String id;

  public IntEx2(String id) { this.id = id; }

  @Override
  public void run() {
    int i=0;
    while ( !Thread.interrupted() ) {
      System.out.println (id+": "+i);
      try {
        Thread.sleep(3000);
      }
      catch (InterruptedException e) {
        System.out.println ("Int while sleeping");

      }
      i++;
    }
  }

  public static void main(String args[]) throws InterruptedException {
    Thread t1 = new Thread( new IntEx2("TEx1") );
    t1.start();
    Thread.sleep(5000);
    t1.interrupt();
  }
}
```

Cuando se interrumpe está dormida y se ejecuta el catch. Tras esto, la hebra ya está activa y no vuelve a interrumpirse

Interrumpir a una Hebra

```
public class IntEx2 implements Runnable {
  private String id;

  public IntEx2(String id) { this.id = id; }

  @Override
  public void run() {
    int i=0;
    while ( !Thread.interrupted() ) {
      System.out.println (id+": "+i);
      try {
        Thread.sleep(3000);
      }
      catch (InterruptedException e) {
        System.out.println ("Int while sleeping");
        Thread.currentThread().interrupt();
      }
      i++;
    }
  }

  public static void main(String args[]) throws InterruptedException {
    Thread t1 = new Thread( new IntEx2("TEx1") );
    t1.start();
    Thread.sleep(5000);
    t1.interrupt();
  }
}
```

Se interrumpe a si misma para poder salir del bucle. Se consigue lo mismo con una variable booleana.

Interrumpir a una Hebra

```
public class IntEx3 implements Runnable {
  public void run() {
    Thread t2 = new Thread() {
      public void run() {
        try { sleep(8000); } catch (InterruptedException e) { }
        System.out.println("T2 is done!");
      }
    };
    t2.start();

    try {
      t2.join();
    } catch (InterruptedException e) {
      System.out.println("T1 was interrupted!");
    }

    System.out.println("T1 is done!");
  }

  public static void main(String args[]) throws InterruptedException {
    Thread t1 = new Thread(new IntEx3());
    t1.start();
    Thread.sleep(5000);
    t1.interrupt();
    System.out.println("Main is done!");
  }
}
```

Accesos simultáneos a memoria

- Qué escribe este programa?

```
public class Test extends Thread {
  static List f = new List();
  private int n;
  private String id;

  public Test(int n, String id) { this.n = n; this.id = id; }

  @Override
  public void run() {
    System.out.println("- Starting '"+id+"'");
    for (int i = 0; i < this.n; i++) {
      System.out.println("Hebra " + this.id + " adds " + i);
      f.add(i);
    }
  }

  static public void main(String[] args) throws InterruptedException {
    Thread t1 = new Test(3, "T1");
    Thread t2 = new Test(3, "T2");
    t1.start();
    t2.start();
    t1.join();
    t2.join();
    System.out.println("Length: " + f.length());
  }
}
```

Accesos simultáneos a memoria

- Qué escribe este programa?

```
public class Test extends Thread {
  static List f = new List(); // atributo compartido por las hebras t1, t2
  private int n;
  private String id;

  public Test(int n, String id) { this.n = n; this.id = id; }

  @Override
  public void run() {
    System.out.println("- Starting '"+id+"'");
    for (int i = 0; i < this.n; i++) {
      System.out.println("Hebra " + this.id + " adds " + i);
      f.add(i);
    }
  }

  static public void main(String[] args) throws InterruptedException {
    Thread t1 = new Test(3, "T1");
    Thread t2 = new Test(3, "T2");
    t1.start();
    t2.start();
    t1.join();
    t2.join();
    System.out.println("Length: " + f.length());
  }
}
```

Accesos simultáneos a memoria

- Qué escribe este programa?

```
public class Test extends Thread {
  static List f = new List(); // atributo compartido por las hebras t1, t2
  private int n;
  private String id;

  public Test(int n, String id) { this.n = n; this.id = id; }

  @Override
  public void run() {
    System.out.println("- Starting '"+id+"'");
    for (int i = 0; i < this.n; i++) {
      System.out.println("Hebra " + this.id + " adds " + i);
      f.add(i);
    }
  }

  static public void main(String[] args) throws InterruptedException {
    Thread t1 = new Test(3, "T1");
    Thread t2 = new Test(3, "T2");
    t1.start();
    t2.start(); // t1 y t2 añaden elementos a f en un orden indeterminado
    t1.join();  // si ambas acceden a la vez es posible que solo una de ellas inserte
    t2.join();
    System.out.println("Length: " + f.length());
  }
}
```

Accesos simultáneos a memoria

- Qué escribe este programa?

```
public class Test extends Thread {
  static List f = new List(); // atributo compartido por las hebras t1, t2
  private int n;
  private String id;

  public Test(int n, String id) { this.n = n; this.id = id; }

  @Override
  public void run() {
    System.out.println("- Starting '"+id+"'");
    for (int i = 0; i < this.n; i++) {
      System.out.println("Hebra " + this.id + " adds " + i);
      f.add(i);
    }
  }

  static public void main(String[] args) throws InterruptedException {
    Thread t1 = new Test(3, "T1");
    Thread t2 = new Test(3, "T2");
    t1.start();
    t2.start(); // t1 y t2 añaden elementos a f en un orden indeterminado
    t1.join();  // si ambas acceden a la vez es posible que solo una de ellas inserte
    t2.join();
    System.out.println("Length: " + f.length()); // la longitud no siempre será 6!!
  }
}
```

Accesos simultáneos a memoria

- Cuando dos hebras que están ejecutando a la vez intentan **modificar un recurso común** simultáneamente, este recurso puede quedar en un estado indeterminado.
- Para evitar este tipo de problemas, se puede usar un **mecanismo de sincronización** para que solo una hebra pueda acceder al recurso a la vez o asegurarse de que todas las modificaciones provienen de una sola hebra.
- El mecanismo de sincronización nos permite declarar que solo una hebra puede ejecutar un bloque de código. Si otra hebra intenta ejecutarlo mientras la primera lo está ejecutando, la segunda hebra se bloquea hasta que acabe la primera.
- Hay varias formas de sincronización en Java.

Modelo de memoria en Java

- Aparte del problema de dejar recursos comunes en estado indeterminado cuando dos hebras acceden simultáneamente al mismo, hay que tener en cuenta que Java utiliza un **Weak memory model**
- Brevemente, un modelo de memoria establece lo que **cada hebra ve cuando se accede a una variable compartida** (atributo).
- Una hebra puede utilizar una memoria caché local en lugar de la memoria principal. Por lo tanto puede ocurrir que las modificaciones a los recursos compartidos no sean visibles a la hebra en el momento que ella accede al recurso. Para que la memoria principal sea visible a todas las hebras de forma correcta, el programa tiene que estar bien sincronizado.
- Veremos el uso de **volatile**, que hace que una hebra siempre tenga acceso al valor real de un atributo.

Modelo de memoria en Java

- Termina el siguiente programa?

```
public class MemoryModelEx1 {
  static boolean f = false;

  public static void main(String[] args) throws InterruptedException {
    new Thread() {
      public void run() {
        System.out.println ("Thread is Starting ");
        while (!f) {
          // do something
        }
        System.out.println ("Thread is Finishing ");
      };
    }.start ();

    Thread.sleep(5000);
    System.out.println ("Setting f to 'true'");
    f = true;
  }
}
```

Modelo de memoria en Java

- Termina el siguiente programa?

```
public class MemoryModelEx1 {
  static boolean f = false;

  public static void main(String[] args) throws InterruptedException {
    new Thread() { // usa su propia memoria con el valor f=false
      public void run() {
        System.out.println("Thread is Starting");
        while (!f) { // el bucle por tanto no termina
          // do something
        }
        System.out.println("Thread is Finishing");
      };
    }.start();

    Thread.sleep(5000);
    System.out.println("Setting f to 'true'");
    f = true; // aunque la hebra de "main" ponga f = true, la otra hebra
  } // no detecta el cambio.
}
```

Modelo de memoria en Java

- Termina el siguiente programa?

```
public class MemoryModelEx1 {
  volatile static boolean f = false;

  public static void main(String[] args) throws InterruptedException {
    new Thread() {
      public void run() {
        System.out.println ("Thread is Starting");
        while (!f) {
          // do something
        }
        System.out.println ("Thread is Finishing");
      };
    }.start ();

    Thread.sleep(5000);
    System.out.println ("Setting f to 'true'");
    f = true;
  }
}
```

Modelo de memoria en Java

- Termina el siguiente programa?

```
public class MemoryModelEx1 {
  volatile static boolean f = false;

  public static void main(String[] args) throws InterruptedException {
    new Thread() { // usa la memoria real
      public void run() {
        System.out.println ("Thread is Starting ");
        while (!f) { // el bucle termina cuando main cambia f a true
          // do something
        }
        System.out.println ("Thread is Finishing ");
      };
    }.start ();

    Thread.sleep(5000);
    System.out.println ("Setting f to 'true'");
    f = true;
  }
}
```

Código seguro ante hebras

- Decimos que un código, por ejemplo, una librería de Java, es **Thread Safe** si se puede usar en programas multi-hebras sin riesgo de tener problemas al usar este código simultáneamente en varias hebras a la vez.
- Muchas de las librerías de Java no son **Thread Safe**, ya que sincronizar es una operación muy costosa en tiempo de ejecución.
- Tiene que ser el programador quien decica si quiere utilizar sincronización en el código.
- No todas las coleeciones(ArrayList, HashMap, etc.) son **Thread Safe**. Las versiones **ThreadSafe** se encuentran en **java.util.concurrent**.
- **Swing tampoco es Thread Safe!**

Swing y Hebras

- Swing y AWT crean una nueva hebra que se utiliza para procesar, entre otras cosas, los eventos de entrada (teclado, ratón, etc.)– **Event Dispatch Thread**
- Para garantizar una interacción fluida, la hebra de eventos tiene más prioridad (6).
- La hebra de eventos se encarga también de llamar a los observadores (**actionPerformed**, etc.) cuando sea necesario. Esto implica que el código incluido en los observadores se ejecuta en la hebra de swing.
- Si el procesado del evento tarda mucho tiempo, los componentes de swing dejarán de responder, porque la hebra de eventos está ocupada manejando el último evento (la llamada a **actionPerformed**, etc.).

Swing y Hebras

- Para evitar este tipo de problemas, si el procesamiento de un evento puede tardar mucho, entonces el código correspondiente debe ser ejecutado en otra hebra.
- **Swing** no es **Thread Safe**, por lo tanto todas las modificaciones de objetos **Swing** deben hacerse desde la hebra de **Swing**.
- Por ello para utilizar componentes de **Swing** desde otra hebra, se debe utilizar:

  SwingUtilities.invokeLater(Runnable)

  para que la ejecución se haga de forma ordenada.

Swing y Hebras

- Al inicio de una GUI, se debe mostrar la ventana principal con **invokeLater**:

  SwingUtilities.invokeLater(new Runnable() public void run() VistaSwing v = new VistaSwing(c););

- El método **inbokeLater** es asíncrono: no espera a la terminación del método **run** del objeto **Runnable** pasado como argumento, sino que termina inmediatamente.
- Para ejecutar algo en la hebra de **Swing** y esperar a que termine el métod, se debe utilizar:

  SwingUtilities.invokeAndWait(Runnable r)

- Se puede saber si estamos ejecutando en la hebra de Swing con:

  boolean SwingUtilities.isEventDispatchThread()